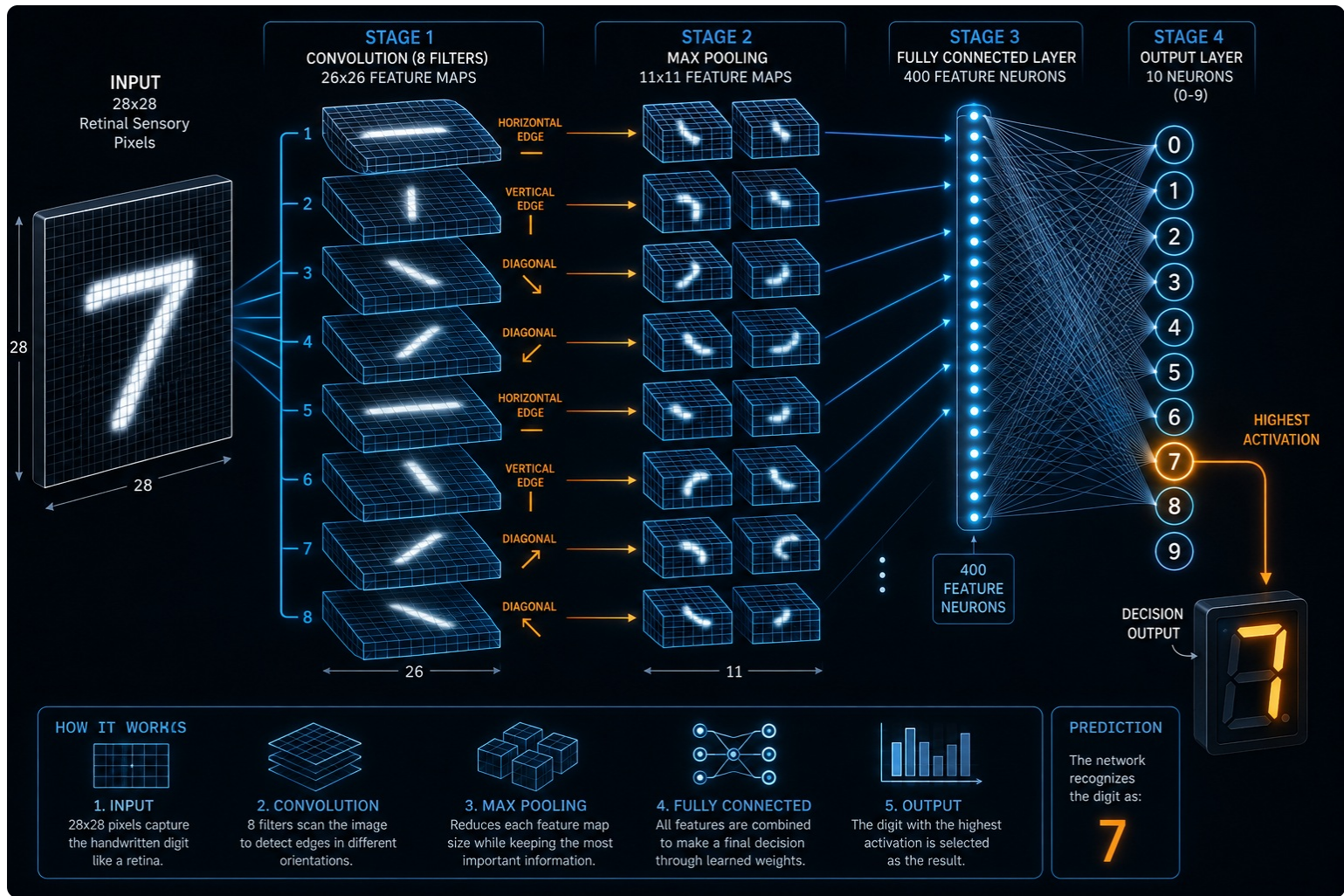


8-bit Fixed-Point 2-Layer CNN

Hardware Accelerator

A beginner-friendly walkthrough of what happens from the incoming 28×28 digit to the final 7-segment prediction — with special focus on the 3-row line buffer and the 9-MAC hardware.



Project in one sentence: a PC sends 784 image bytes over UART; the Basys 3 FPGA stores them, runs the complete fixed-point CNN in dedicated hardware, selects the highest-scoring digit, and drives the 7-segment display.

This guide walks through the design in seven short sections: the full dataflow, a from-scratch explanation of the 3-row line buffer, the 9-MAC parallel compute array, the convolution engine's finite-state machine, simulation vs. real hardware, why the architecture is efficient, and a one-picture summary of the entire pipeline.

Quick fact	Value
Target board	Digilent Basys 3 (Xilinx Artix-7 FPGA)
System clock	100 MHz
Image link	UART, 115,200 baud, 784 bytes per frame
Numeric format	8-bit fixed-point (INT8) weights and activations
Network	2 convolution layers + 2 max-pool layers + 1 fully-connected layer
Inference latency	≈1019 clock cycles ≈ 10.19 μs (Conv1 through Argmax)
Output	Predicted digit (0-9) on a 7-segment display

1. Complete Hardware Dataflow

The table below is the map of the whole design. Each row is one stage the image passes through, in order, from the raw pixels arriving over UART to a digit lighting up on the display.

Stage	Shape / operation	What it does
Input	28×28 = 784 pixels	Stores the grayscale handwritten digit in Image RAM.
Conv1	26×26×8	8 filters detect basic edges and line orientations.
Pool1	13×13×8	2×2 max-pooling keeps strong activations while reducing size.
Conv2	11×11×16	16 filters combine earlier edges into more complex shapes.
Pool2	5×5×16 = 400	Produces 400 compact feature values.
FC	400 → 10	Computes ten class scores, one for each digit 0–9.
Argmax	10 scores → 1	Chooses the class with the largest score.
Display	Digit 0–9	The winning digit is shown on the physical 7-segment display.

How to read this table: every stage takes the output shape of the previous one as its input. Notice how the "area" of the image (28×28, then 26×26, then 13×13...) keeps shrinking while the "depth" (1, then 8, then 16) grows — the hardware is trading spatial detail for a richer, more compact description of what the digit looks like.

What happens inside the CNN

The CNN is not executing as a software program. Its operations are implemented as Verilog hardware blocks controlled by a master finite-state machine (FSM). The same RTL logic is used in simulation and on the physical FPGA; only the environment changes.

- **Input:** the PC sends 784 bytes through UART at 115,200 baud. The UART receiver stores each byte in Image RAM. After all 784 bytes arrive, the master FSM starts inference.
- **Conv1:** a 3×3 kernel moves across the image. Each of the 8 filters produces a 26×26 feature map, responding to simple local patterns such as horizontal, vertical, or diagonal edges.
- **MaxPool1:** every 2×2 region is reduced to its maximum value, giving 13×13×8. This shrinks the data later hardware must process while keeping the strongest features.
- **Conv2:** operates on the 8 feature channels and produces 16 maps of size 11×11 — learned combinations that can respond to corners, curves, and intersections.
- **MaxPool2:** produces 5×5×16 = 400 values, which flatten directly into the 400 inputs of the fully-connected layer.
- **FC and Argmax:** the FC stage computes ten scores from the 400 features, one per digit class. Argmax picks the largest score, which is latched for display.

Fixed-point detail: the accelerator uses INT8 weights and activations rather than floating-point values. After convolution, the wider accumulator is right-shifted (SHIFT = 9 for Conv1 and SHIFT = 8 for Conv2 in the supplied design), then ReLU and 8-bit saturation keep the stored feature values compact.

Why INT8 instead of floating point?

Floating-point multipliers are large, power-hungry, and mostly unnecessary for a task like digit recognition, where small numeric errors barely affect the final classification. By restricting every weight and activation to an 8-bit signed integer, the whole network fits comfortably in the FPGA's DSP48E1 blocks and BRAM, and every multiply becomes a single-cycle integer operation instead of a multi-cycle floating-point one. The trade-off is a small, usually imperceptible loss of numeric precision compared to a full floating-point model — a standard technique called **quantization**, widely used in real-world embedded and edge-AI accelerators for exactly this reason.

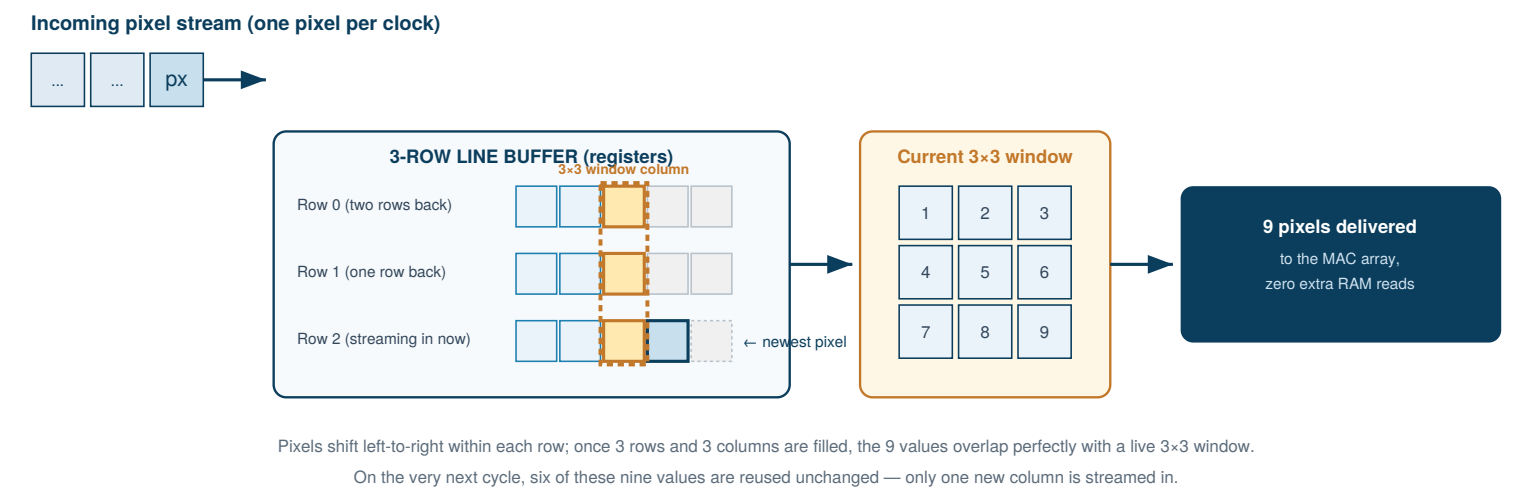
2. The Line Buffer, Explained From Scratch

The problem it solves: a 3×3 convolution needs a small 3-row-by-3-column window of the image at every step. But the image arrives as a single stream of pixels, one at a time, row after row — not as a neat little square. So the hardware needs a way to turn a 1-pixel-at-a-time stream into a 3×3 square it can hand to the multipliers.

Everyday analogy: imagine reading a book one letter at a time through a tiny slot, but you need to see a 3×3 block of letters spanning three lines to understand each "word." You can't jump back three lines every time — that's too slow. Instead, you keep the last two lines you've already read sitting in memory, and only the newest letter needs to be added. The line buffer is exactly this trick, done in hardware.

Why not just re-read pixels from memory?

The naive approach would be: for every output position, go fetch all nine neighboring pixels from RAM. But neighboring 3×3 windows overlap heavily — when the window slides one pixel to the right, six of the nine pixels are the exact same ones used in the previous window. Re-fetching them from memory every single time wastes memory bandwidth and clock cycles. The line buffer's job is to **keep those shared pixels sitting in fast registers** so they never need to be fetched twice.



Why this matters for speed: RAM reads are relatively slow and power-hungry compared to reading a value already sitting in a register. A design that re-read all nine pixels from Image RAM for every single output position would need far more memory bandwidth than one that streams pixels once and reuses them in place. The line buffer converts a memory-bandwidth problem into a small, cheap set of shift registers.

A concrete walk-through

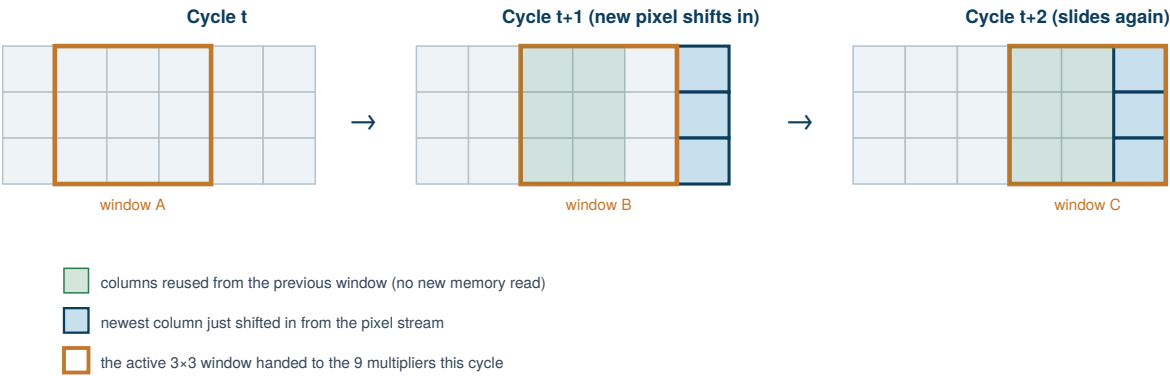
Say the image is being scanned left-to-right, top-to-bottom (this is called "raster order"). Label pixel positions by (row, column), starting at (0,0) in the top-left corner:

- While rows 0 and 1 are still streaming in, **no 3×3 window can be formed yet** — there simply aren't three rows available. The line buffer is just filling up.
- The moment column 2 of row 2 arrives, the hardware has seen rows 0, 1, and 2, and columns 0 through 2 of each. That is exactly nine pixels forming a valid 3×3 window centered on (1,1) — the very first convolution output can now be computed.
- As row 2 keeps streaming (column 3, column 4, ...), each new pixel completes another valid window one column to the right, without ever re-reading row 0 or row 1 from memory.
- When row 2 ends and row 3 begins, the data that was in Row 0 is no longer needed by any future window, so it is safely overwritten to become the buffer for the new incoming row — this is the row rotation described on the next page.

2. The Line Buffer — Step by Step

Here is exactly what happens, one clock cycle at a time, as pixels stream in:

1. **Three row buffers.** The hardware keeps three small shift-register rows: **Row 0** (the oldest row still needed), **Row 1** (the row above the current one), and **Row 2** (the row currently streaming in).
2. **A new pixel arrives.** Every clock cycle, one new pixel is shifted into **Row 2** from the left side.
3. **Older pixels shift along, not away.** As the new pixel enters, the row buffers act like a bucket brigade: pixels move one position to the right inside the register, but nothing is thrown away until it's no longer needed. Pixels from the two previous rows stay parked in **Row 0** and **Row 1**.
4. **The window "appears" for free.** Once three full rows and three columns have been seen, the same three columns of **Row 0**, **Row 1**, and **Row 2** can be read out *simultaneously* — this is the 3×3 window, ready to hand to the 9 multipliers, with zero extra memory reads.
5. **Slide by one.** On the next clock cycle, a new pixel shifts in and the whole window slides one column to the right. Six of the nine values are simply reused from the previous cycle; only the newest column is new.
6. **Row rotation.** When a row is finished, the buffer's role rotates: what was **Row 1** becomes the new **Row 0**, what was **Row 2** becomes the new **Row 1**, and a fresh row starts filling **Row 2**.



Each step, the window slides one column right. Two of every three columns carry over unchanged — only one new column of pixels needs to enter the buffer, which is the whole point of the design.

Important intuition: the line buffer is mainly a *data-reuse* mechanism. It does not change the mathematical convolution at all — the multiplications and additions are identical to the naive approach. What it changes is *how efficiently* the same nine pixels are delivered to the arithmetic hardware, cycle after cycle, without repeated trips to memory.

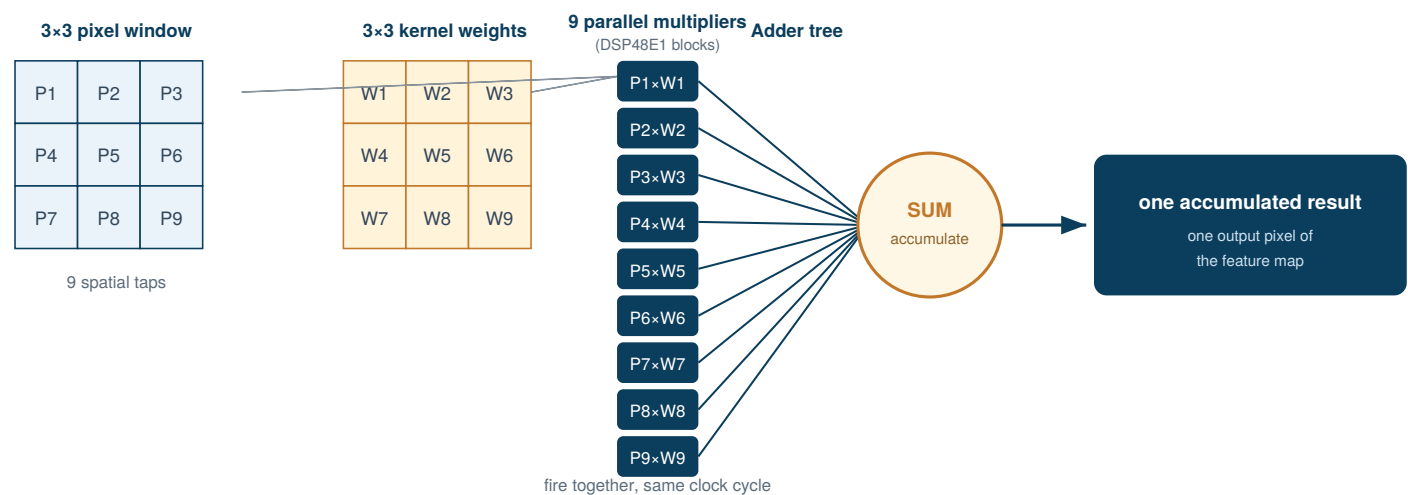
Why only 3 rows, not the whole image? At any instant, a 3×3 convolution only ever needs pixels from the current row and the two rows above it — nothing higher up in the image is ever touched again. So storing exactly three rows is the minimum needed to guarantee every future window can be built without re-reading RAM. This is why the buffer is called a "3-row" line buffer rather than a full-frame buffer, and why it uses so few hardware resources despite handling a full-resolution image stream.

The hardware cost of a line buffer

Each row buffer only needs to be as wide as one row of the image (28 pixels for the input, fewer for later, smaller feature maps), and each pixel is stored in a small 8-bit register. So the entire 3-row buffer costs roughly $3 \times 28 \times 8 = 672$ flip-flops for the input stage — a tiny fraction of the FPGA's total register budget. This is the key reason the technique scales so well: the buffer size grows only with image *width*, never with image *height*, no matter how tall the incoming frame is.

3. The 9-MAC Parallel Compute Array

Once a 3×3 window is ready, convolution is just a dot product: each of the 9 pixels is multiplied by its corresponding kernel weight, and the nine products are added together. A "MAC" is short for **Multiply-ACcumulate** — one multiply followed by one running addition. The accelerator performs all nine multiplications *at the same time*, in parallel hardware, rather than one after another.



Why this is fast: a sequential design with a single multiplier would perform the nine multiply operations one after another — nine steps. Here, nine physical DSP48E1 multiplier blocks fire together for the nine spatial taps, so the arithmetic portion of one 3×3 window collapses from roughly nine multiply steps down to a single parallel multiply step, followed by a fast adder tree that combines the nine products into one accumulated sum.

But it isn't 9× faster everywhere: the supplied RTL still processes input channels and output filters sequentially. For Conv2, the 8 input channels require separate weight-load-and-multiply passes, so the FSM introduces overhead around the highly parallel MAC stage. The 9× speedup applies to the innermost spatial dot product, not to the full convolution across every channel and filter.

Where the numbers come from: the DSP48E1 is a hardened arithmetic block built directly into the Artix-7 fabric. Because it already contains a multiplier and adder in silicon, using it is far cheaper — in both area and power — than building an equivalent multiplier out of general-purpose logic gates.

A worked numeric example

Suppose a 3×3 window has pixel values [10, 20, 30, 40, 50, 60, 70, 80, 90] and the loaded kernel weights are [1, 0, -1, 1, 0, -1, 1, 0, -1] (a simple vertical-edge detector). The nine multiplications happen in one parallel step:

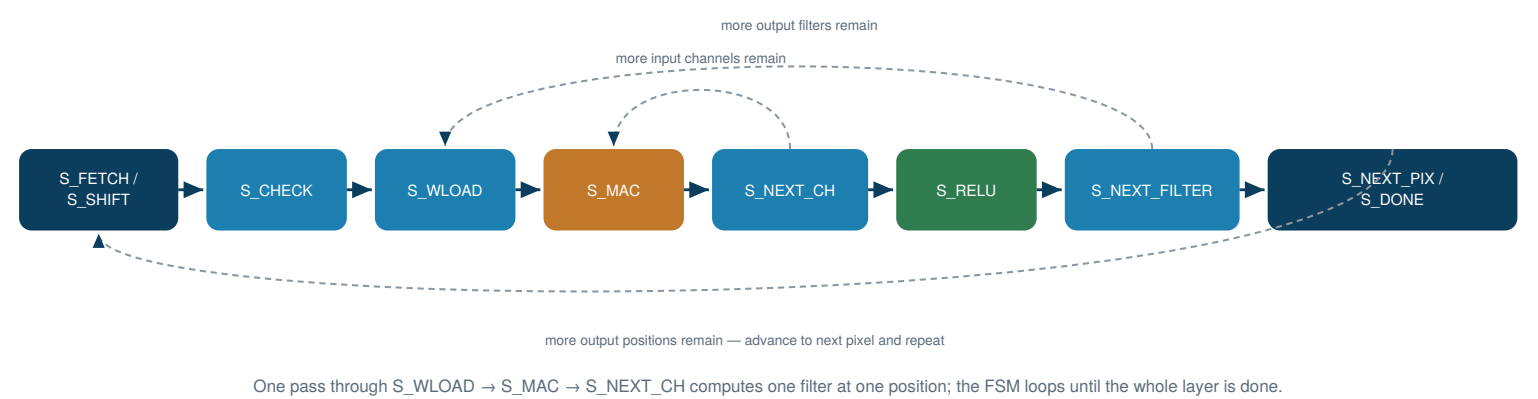
Tap	Pixel × Weight	Product
1-3	10×1, 20×0, 30×(-1)	10, 0, -30
4-6	40×1, 50×0, 60×(-1)	40, 0, -60
7-9	70×1, 80×0, 90×(-1)	70, 0, -90

The adder tree sums all nine products: 10+0-30+40+0-60+70+0-90 = **-60**. That raw sum is then right-shifted (SHIFT = 9 for Conv1), passed through ReLU (which clamps negative values to zero), and saturated to 8 bits before being stored as one pixel of the output feature map.

4. What the Conv Engine Actually Does

The whole convolution engine is driven by one finite-state machine (FSM) that walks through the following states for every output position. Think of it as a checklist the hardware repeats, over and over, until the entire feature map has been computed:

FSM state	Purpose
S_FETCH → S_SHIFT	Read the next raster pixel and capture it into the line buffer.
S_CHECK	Check whether a complete 3×3 window exists yet.
S_WLOAD	Load the 9 weights for the current filter/channel pair.
S_MAC	Fire the 9 DSP multipliers and accumulate the products.
S_NEXT_CH	Move to the next input channel when required.
S_RELU	Apply ReLU, quantize/shift, and prepare the output.
S_NEXT_FILTER	Repeat for the next output filter.
S_NEXT_PIX / S_DONE	Advance to the next output position or finish the layer.



Reading the loop-back arrows: the dashed arrows above show where the FSM repeats work. It loops back to `S_WLOAD` while input channels remain, loops back to `S_CHECK` while output filters remain, and loops back to `S_FETCH / S_SHIFT` while output positions remain. Only once all three loops are exhausted does the layer reach `S_DONE`.

Why Conv2 takes longer than Conv1

Conv1 reads directly from the single-channel input image, so for every output position it only needs one pass through `S_WLOAD → S_MAC` per filter. Conv2, however, reads from **8 input channels** produced by Pool1. For every output position and every one of the 16 output filters, the FSM must loop through `S_NEXT_CH` eight times — once per input channel — accumulating partial sums before moving on. This is exactly the loop-back arrow labeled "more input channels remain" in the diagram above, and it is the main reason Conv2's share of the total 1019-cycle budget is proportionally larger than Conv1's, despite Conv2's output being spatially smaller (11×11 vs. 26×26).

Takeaway: the 9-MAC array makes the innermost spatial dot product fast, but the FSM still has to loop over channels and filters in sequence. Total layer latency is roughly proportional to *output positions × output filters × input channels*, with the 9-wide parallelism only speeding up the innermost dimension.

Approximate cycle-count budget

Stage	Rough share of the ≈1019-cycle inference
Conv1 (8 filters, 26×26 out)	Largest spatial size, but only 1 input channel per filter
Pool1	Small — simple comparisons, no multiplies
Conv2 (16 filters, 11×11 out, 8 channels)	Smaller spatial size, but 8× more channel passes per position
Pool2	Small — simple comparisons, no multiplies
FC (400→10) + Argmax	One dot product per class, then a 10-way compare

Exact per-stage cycle counts depend on the synthesized FSM timing; the table shows the relative cost drivers rather than a precise breakdown.

5. Simulation vs. Real FPGA

Simulation and FPGA execution use the exact same RTL datapath. Simulation provides visibility and functional verification; the FPGA proves that the synthesized design works with real placement, routing, I/O, and hardware timing.

Simulation (XSim)	Physical FPGA (Basys 3)
Testbench generates a 100 MHz clock and reset.	The Artix-7 board supplies the real clock and reset behavior.
Testbench bit-bangs 784 UART bytes at 115200 baud.	A PC sends the image through a real UART connection.
Weights load from .mem files using \$readmemh.	The synthesized design uses the mapped memory resources.
Internal line-buffer, MAC, FSM and RAM signals are visible in waveforms.	External behavior is observed through UART, LEDs and the 7-segment display.
Checks functional correctness before hardware deployment.	Adds real synthesis, placement/routing, DSP/BRAM mapping and I/O behavior.

Why the architecture is efficient

- **Pixel reuse:** the 3-row line buffer avoids repeatedly fetching overlapping pixels from RAM.
- **Spatial parallelism:** 9 MACs process the nine taps of a 3×3 window concurrently.
- **DSP utilization:** the 8-bit multiplications map efficiently onto hardened DSP48E1 blocks.
- **INT8 datapath:** smaller weights and activations reduce storage and arithmetic cost compared with floating point.
- **Deterministic control:** the FSM follows a fixed sequence, so inference latency is predictable.
- **Small FPGA fit:** the supplied design uses about 18 of 90 DSP48E1s, about 6–8 of 50 BRAM tiles, and roughly 3K of 65K flip-flops, leaving substantial device headroom.

Big picture: none of these choices is exotic on its own — line buffers, parallel MACs, and INT8 quantization are all standard techniques in CNN accelerator design. What makes this project work well on a small Basys 3 board is that they are combined carefully enough to fit comfortably inside the Artix-7's limited DSP and BRAM budget while still meeting the timing needed for a 100 MHz clock.

Resource utilization at a glance

Resource	Used	Available on Basys 3	Utilization
DSP48E1 slices	≈18	90	≈20%
Block RAM (BRAM) tiles	≈6–8	50	≈13–16%
Flip-flops	≈3,000	65,000+	≈5%

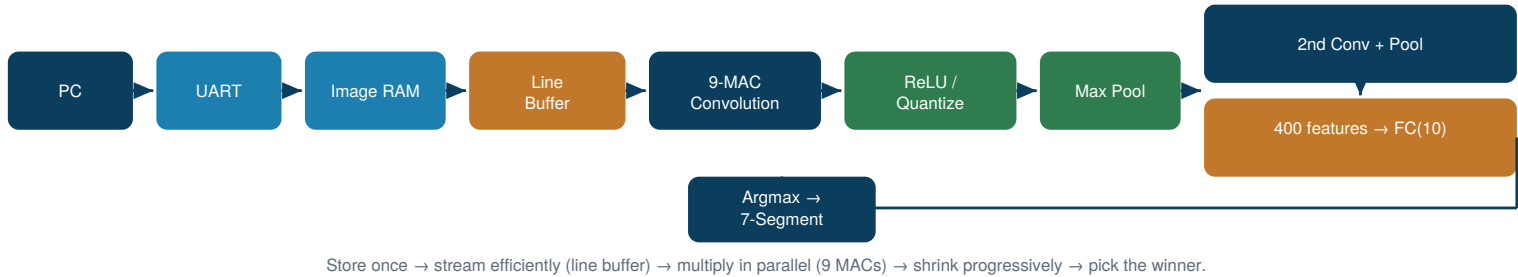
Figures are approximate, taken from the supplied project's synthesis report. The low utilization leaves headroom for adding features such as additional filters, a deeper network, or extra I/O without running out of device resources.

6. Understanding the Two Latency Numbers

The supplied project reports approximately **1019 clock cycles at 100 MHz $\approx$ 10.19 μ s** for the CNN computation from Conv1 through Argmax. This is the meaningful number when evaluating the accelerator's internal compute speed. The complete user-visible round trip also includes sending 784 bytes over 115200-baud UART, which takes tens of milliseconds and therefore dominates the end-to-end delay.

Key distinction: UART latency is communication overhead; 10.19 μ s is the hardware inference latency. They should not be quoted as if they measure the same thing. A helpful comparison: the UART transfer is like the time spent mailing a letter, while the 10.19 μ s inference is like the few seconds it takes the recipient to actually read it once it arrives.

7. The Whole Project in One Mental Picture



The central idea is simple: store the image once, stream pixels efficiently, reuse overlapping data with the line buffer, perform the nine spatial multiplications in parallel, then progressively reduce the feature information until only ten class scores remain.

Recap of the five ideas that make it work: (1) a line buffer avoids re-reading overlapping pixels, (2) nine parallel MACs turn a 3×3 convolution into one clock cycle of arithmetic, (3) INT8 fixed-point math keeps the datapath small, (4) a single FSM sequences everything deterministically, and (5) pooling layers progressively shrink the data so the final classifier only has to weigh 400 numbers instead of the original 784.

Quick glossary

Term	Meaning
RTL	Register-Transfer Level — Verilog code describing hardware behavior in terms of registers and logic.
FSM	Finite-State Machine — a controller that moves through a fixed set of named states.
MAC	Multiply-ACcumulate — one multiplication followed by a running addition.
DSP48E1	A hardened arithmetic block built into the Artix-7 FPGA fabric, used for fast multiply-accumulate.
BRAM	Block RAM — dedicated on-chip memory tiles used to store the image and intermediate data.
ReLU	Rectified Linear Unit — clamps negative values to zero, a standard CNN activation function.
Quantization	Converting values to a smaller fixed-point format (here, 8-bit integers) to save area and power.

Source basis: the supplied CNN_FPGA explainer and the project description provided with this request. Terminology and implementation-specific details are kept aligned with those materials.